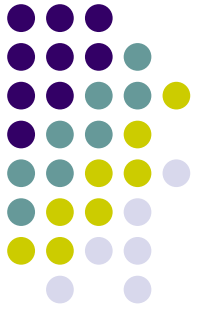


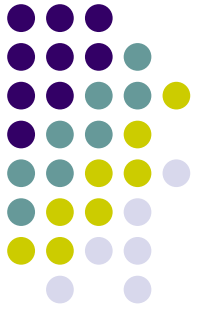
Router (Switch): Intro & Structure



Perspective & Terminology

- **Repeaters/Hubs**: physical level (L1)
- **Bridges/ethernet switches**: datalink level (L2)
 - based on MAC addresses
 - discover attached stations by listening
- **Routers/Switches**: network level (L3)
 - participate in routing protocols
- Application level **gateways**: application layer (L7)
 - treat entire network as a single hop

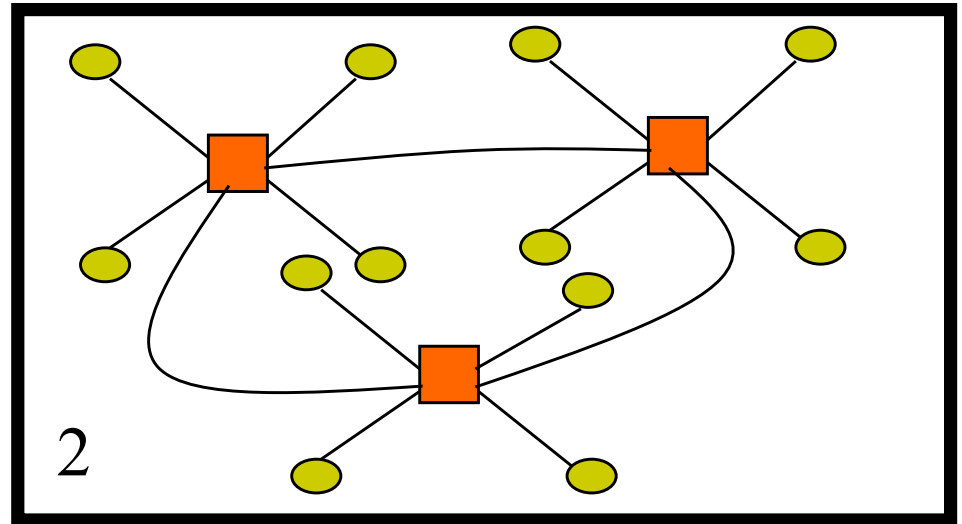
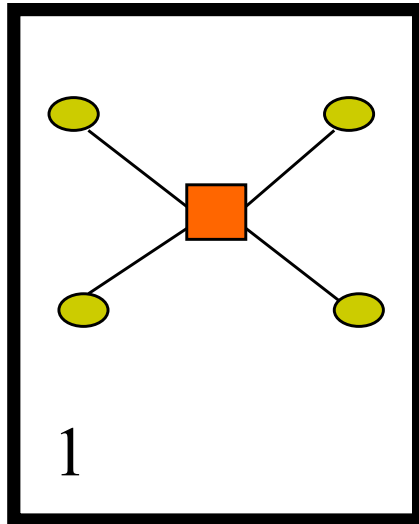
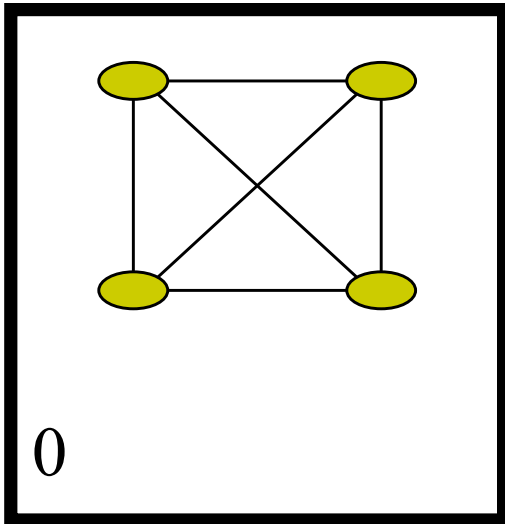
Principle: higher the layer → better functionality, worse performance



What is it all about?

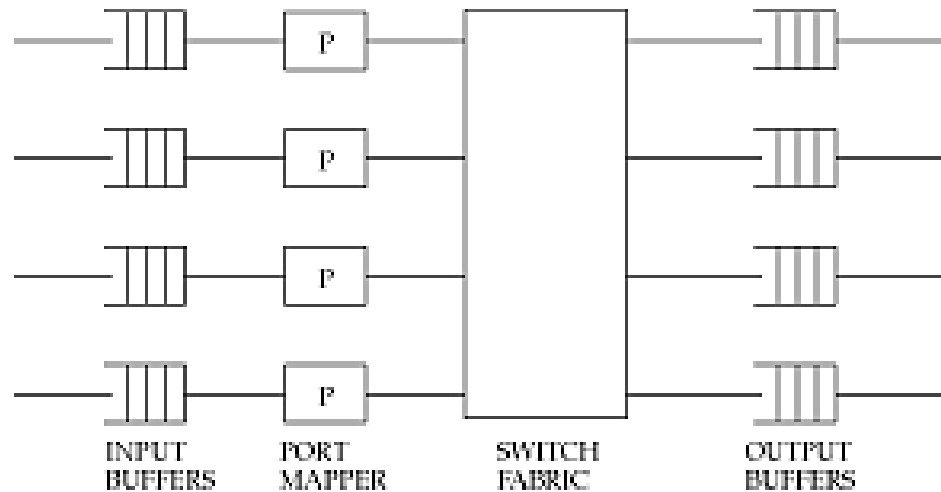
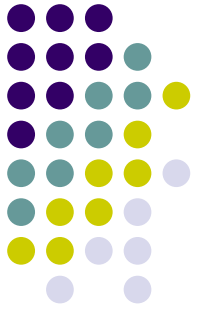
How do we move traffic from one part of the network to another?

- solution (0) connect all pairs: $\Theta(n^2)$ wires, not scalable
- solution (1): connect all to a **switch**
- solution (2): connect switches to each other



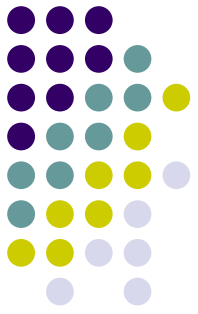
Switch Functionality: Data arriving at an input port is moved to an output port (or ports)

A schematic switch



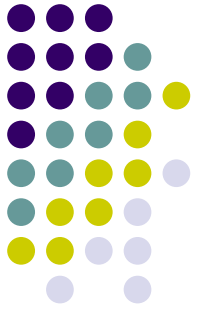
Each element is optional

Switching Strategies



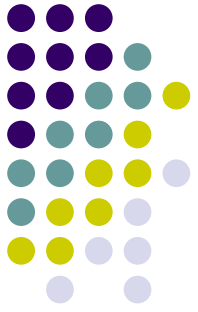
- Circuit switching
 - Dedicated communication circuit (e.g., telephone)
 - Links reserved for use by communication channel
 - Constant rate bit stream
- Packet switching
 - Links used dynamically
 - Send/receive messages (packets)
 - Admission policies and other traffic determine bandwidth





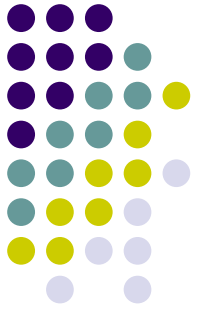
Types of switching elements

- Circuit switching (connection oriented)
 - Telephone switches, moving 8-bit samples
- Packet switching (connection-less)
 - Datagram routers, moving datagrams (packets)
- Packet vs. circuit switches
 - connection oriented switches need call setup
 - setup is handled in *control plane* by *switch controller*
 - connectionless switches deal with *self-contained* datagrams



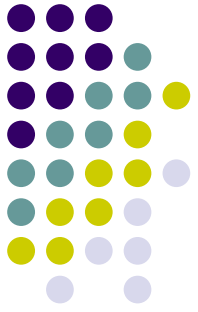
Switch Variants

- In a circuit switch, route of bits is determined at time of connection setup
 - Switch maintains per-flow **state**
 - Packets (cells) don't need headers
- In a packet switch, each packet is routed individually
 - Each packet carries a destination field
 - Switch is unaware of flows: looks up destination port for each packet
- But **forwarding** is very similar in both types



Switching element: Duties

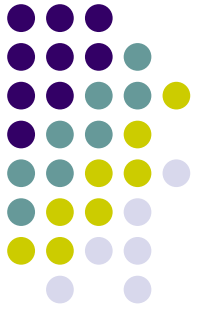
- Primary goal: forwarding
 - May signal to sender about congestion (**backpressure**)
- Participate in (distributed) routing algorithms
 - Input: network state
 - Output: routing tables (which way the destination)
- Resolve contention on output links
 - Input: competing packets/flows
 - Output: Schedules (which packet is transmitted now)
- Admission control
 - Input: connection requests (QoS)
 - Output: admit/reject + resource allocation



Performance criteria

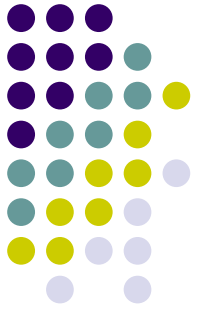
- Primary goal: maximize capacity
 - **Capacity of switch:** max total rate of connections
 - subject to cost and reliability constraints
- Circuit switch goal: minimize call blocking
 - **Call blocking:** if there is no path from input to output
- Packet switch goal: minimize packet loss
 - **Packet loss:** when a packet can't be forwarded or stored
 - Softer requirement: No packet reordering (Local-FCFS)

Blocking in packet switches



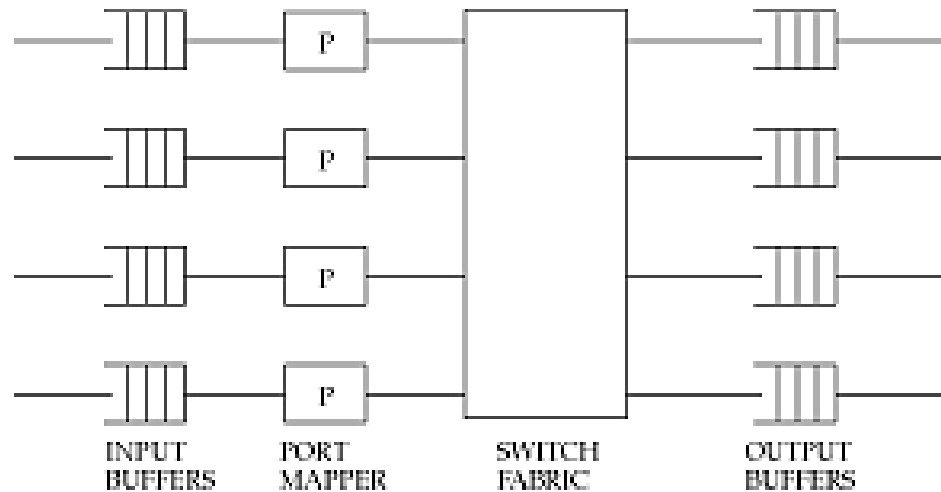
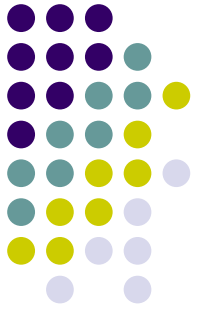
- Blocking: switch can't deliver data to output port
- Can have both **internal** and **output** blocking
- Internal blocking: no path to output
- Output blocking: output link is busy, output buffer full
- If a packet is blocked, it is either **stored** or **dropped**

Strategies for dealing with blocking



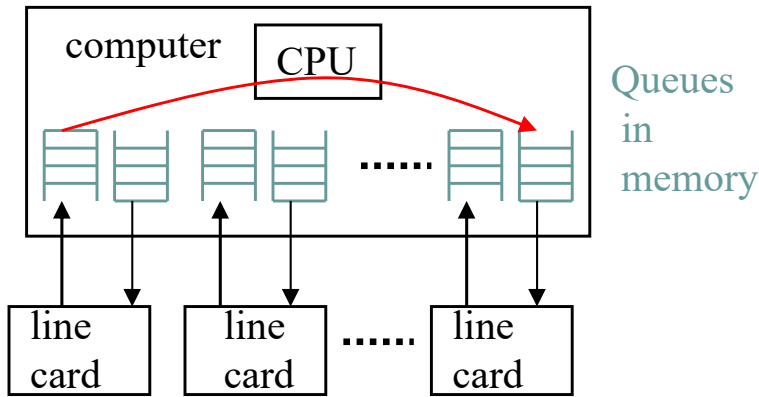
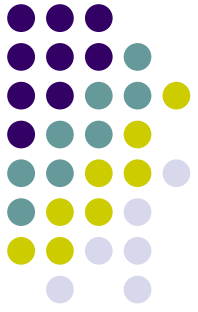
- Avoid internal blocking:
 - **Overprovisioning**: internal links faster than inputs
 - Parallel fabrics
- Wait till clear:
 - Use **buffers**, at input or output
 - Effective if bursts are short
- Let predecessor deal with it:
 - **Backpressure**: don't accept packet until path is available
 - Implies admission control at the source

A schematic switch

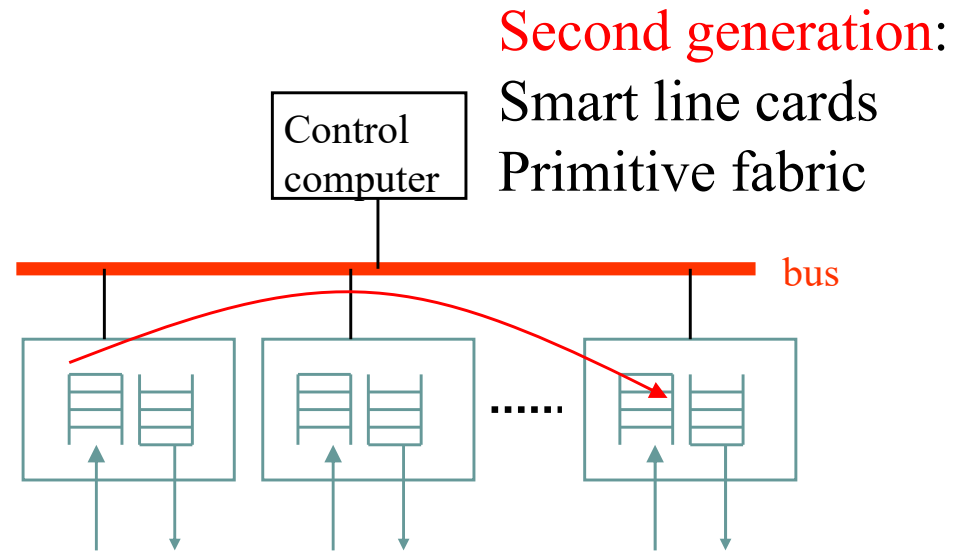


Each element is optional

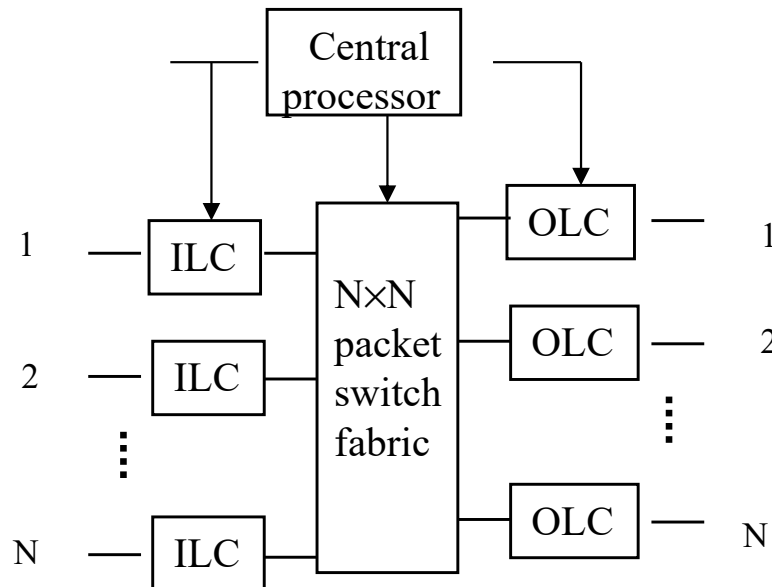
Three generations of packet switches



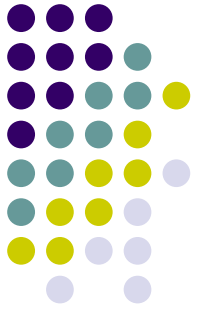
First generation:
Stupid line card
Primitive fabric



Second generation:
Smart line cards
Primitive fabric

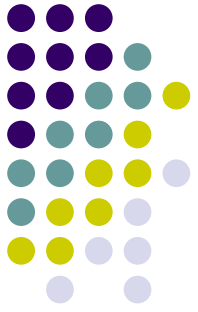


Third generation:
Smart line cards
Sophisticated fabric



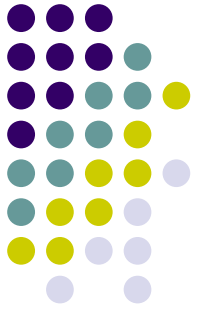
Switching Techniques

- Time slot interchange (TSI)
- Space switching:
 - crossbar
 - TS
 - Fabrics:
 - Clos
 - Benes
 - Recursive constructions



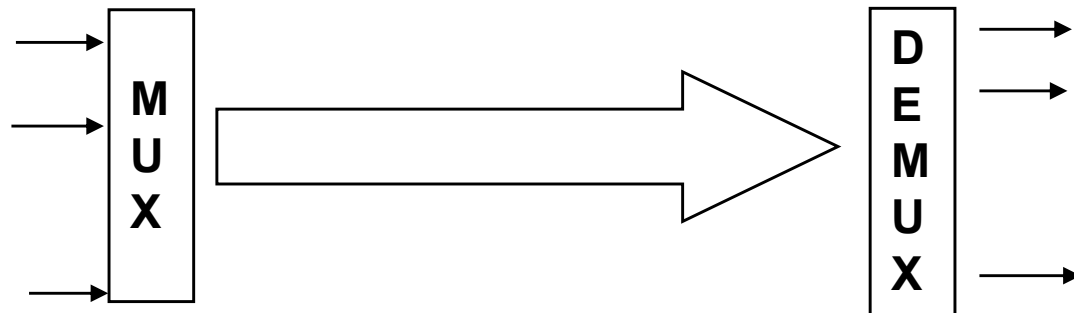
Circuit switching

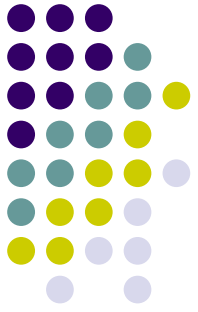
- Telephone: Moving 8-bit samples from an input port to an output port
- Consider something simpler than a switch:
a multiplexer / demultiplexer
 - MUX: merge
 - DEMUX: split



Multiplexers, demultiplexers

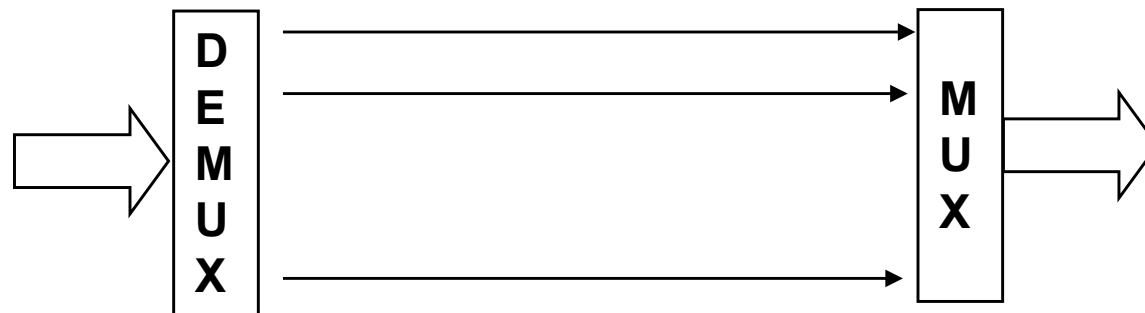
- Synchronous multiplexor
 - N inputs lines, speed 1
 - 1 output line, speed N
- Demultiplexer
 - 1 input line of speed N , N output lines of speed 1
 - Samples are placed in output buffers in round robin order
- Can compose recursively

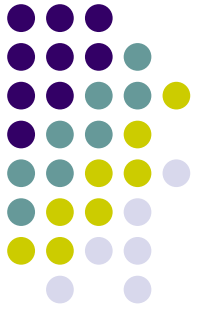




Also: Inverse multiplexing

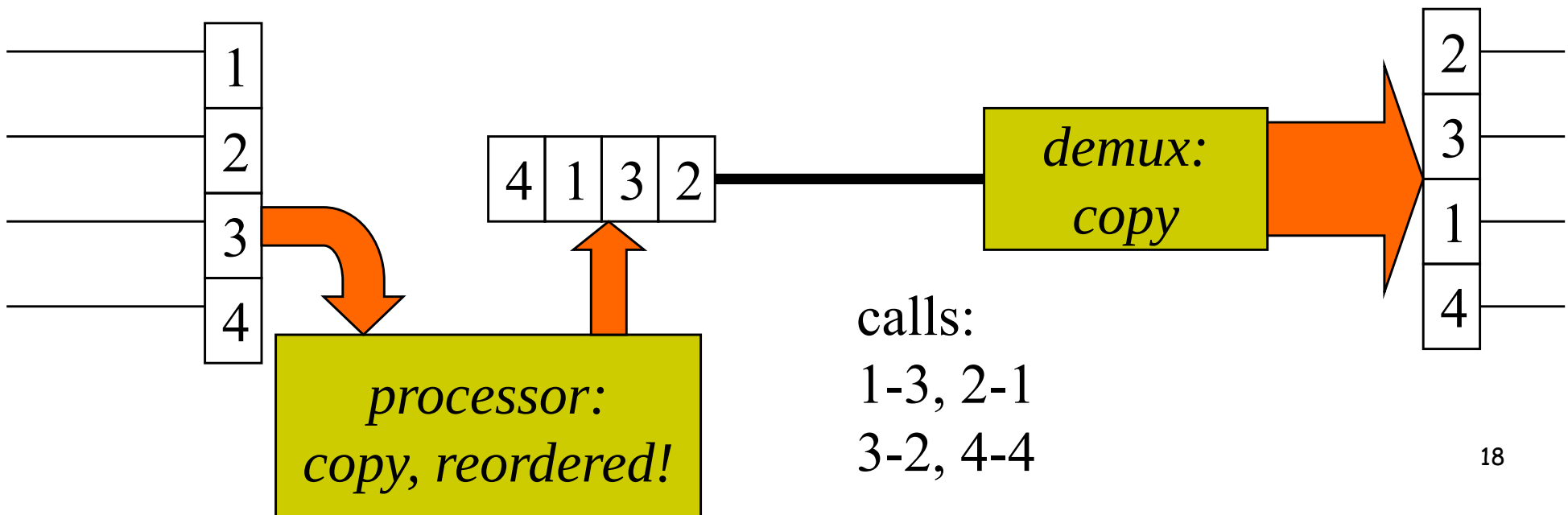
- Takes a high bit-rate stream and scatters it across multiple links
- At the other end, combines multiple streams
 - resequencing to accommodate variation in delays
- Allows high-speed virtual links using existing technology
 - Example: video conferencing over 3 ISDN lines, MultilinkPPP

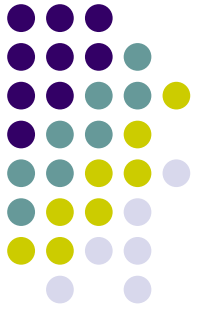




Mux+Demux+Schedule=Switch

- Idea: when demultiplexing, position in frame determines output link
- **Time division switching** interchanges sample position within a frame: time slot interchange (TSI)

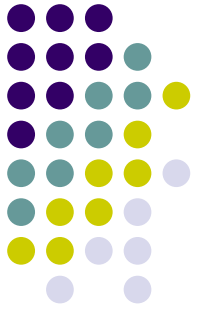




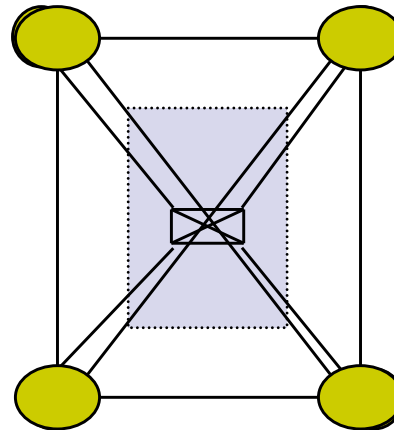
TSI Pros and Cons

- Pro: simple!
 - Can implement with shared memory: write and read to different locations
- Con: limited. Memory access is bottleneck.
- For example: 120,000 telephone circuits
 - Each circuit reads & writes 8 bits every 125 ns.
 - Need $120,000 \times 8000 \times 2 \times 8 > 15\text{Gbps}$ memory bandwidth
- Need to look to other techniques for this scale
 - How is it solved inside the memory chip?

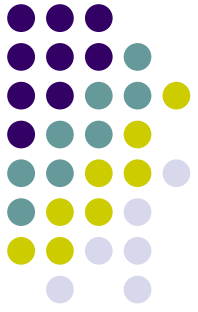
Space division switching



- Idea: allocate a different path from each source to each destination
- Seems like connecting all $n(n - 1)/2$ different pairs, but in a single box

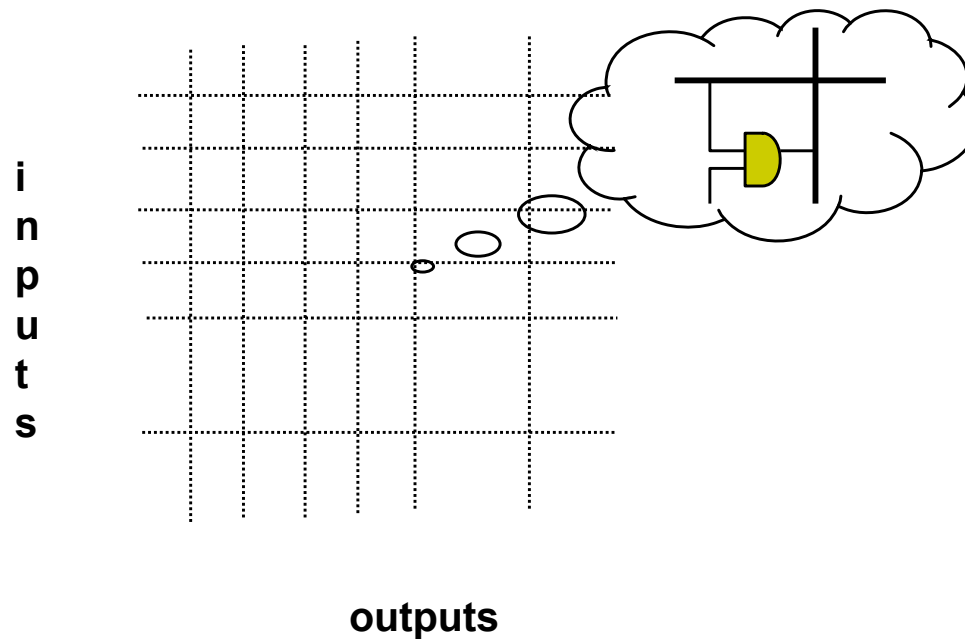


Basic implementation: Crossbar

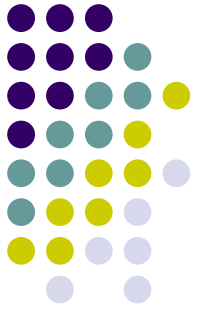


Simple space-division switch

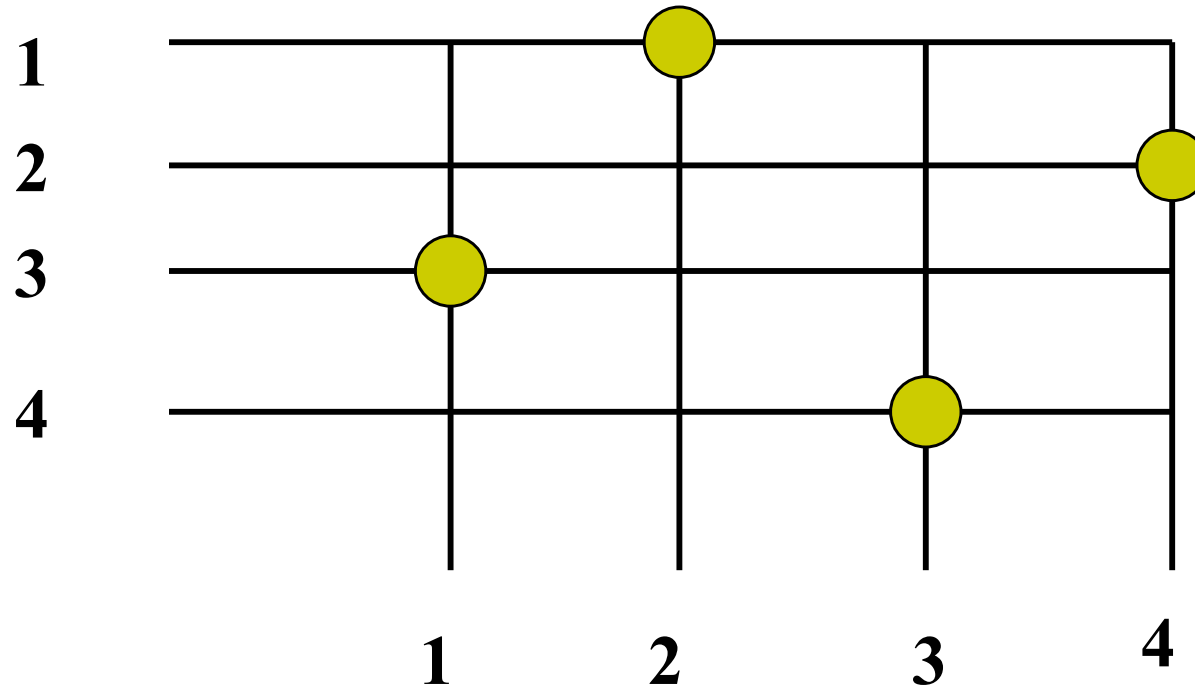
- **Crosspoints** can be turned on or off



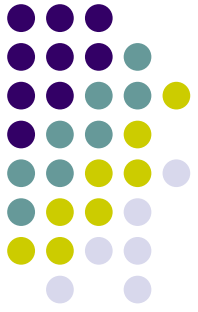
Crossbar - example



sessions: (1,2) (2,4) (3,1) (4,3)

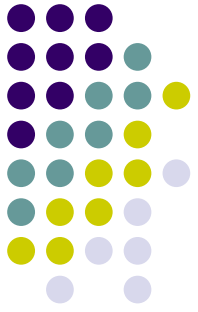


Crossbar

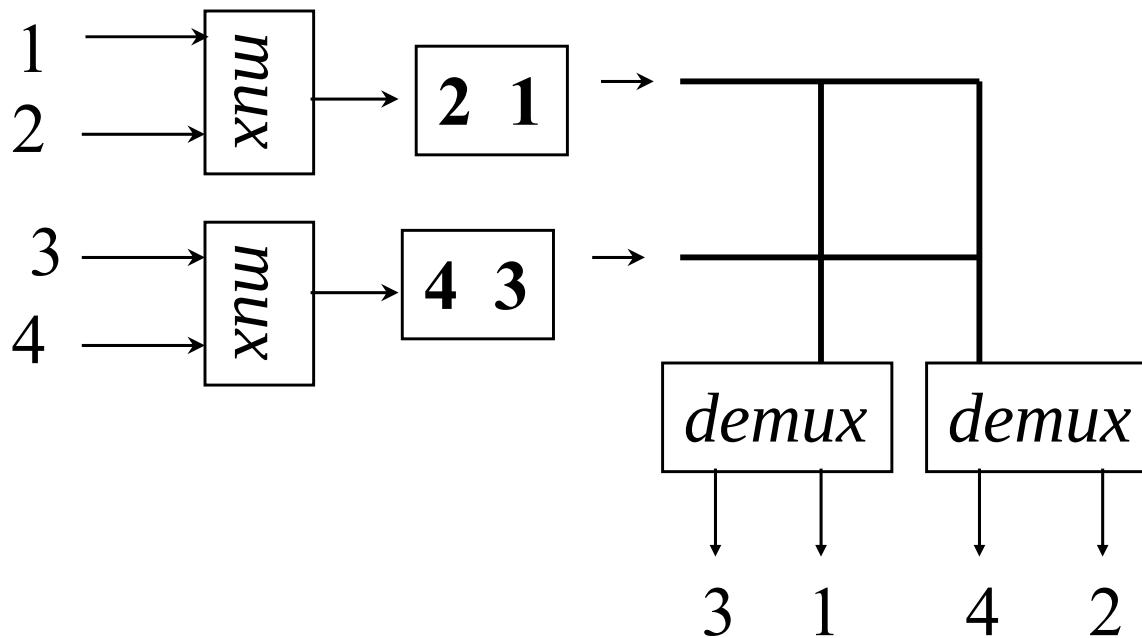


- Advantages:
 - simple to implement
 - simple control
 - flexible
- Drawback: again, limited scale
 - number of crosspoints, N^2
 - large VLSI layout area
 - Bottleneck: number of pins! Need $\log N$ pins for each input port to encode output port
- Bottom line: good for small N (hundreds)

Combination: Time-space switching

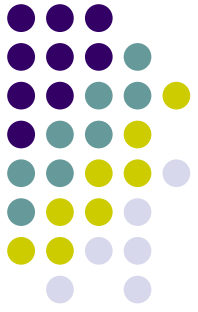


- Precede each input link in a crossbar with a TSI
- Delay samples so that they arrive at the right time for the space division switch's schedule



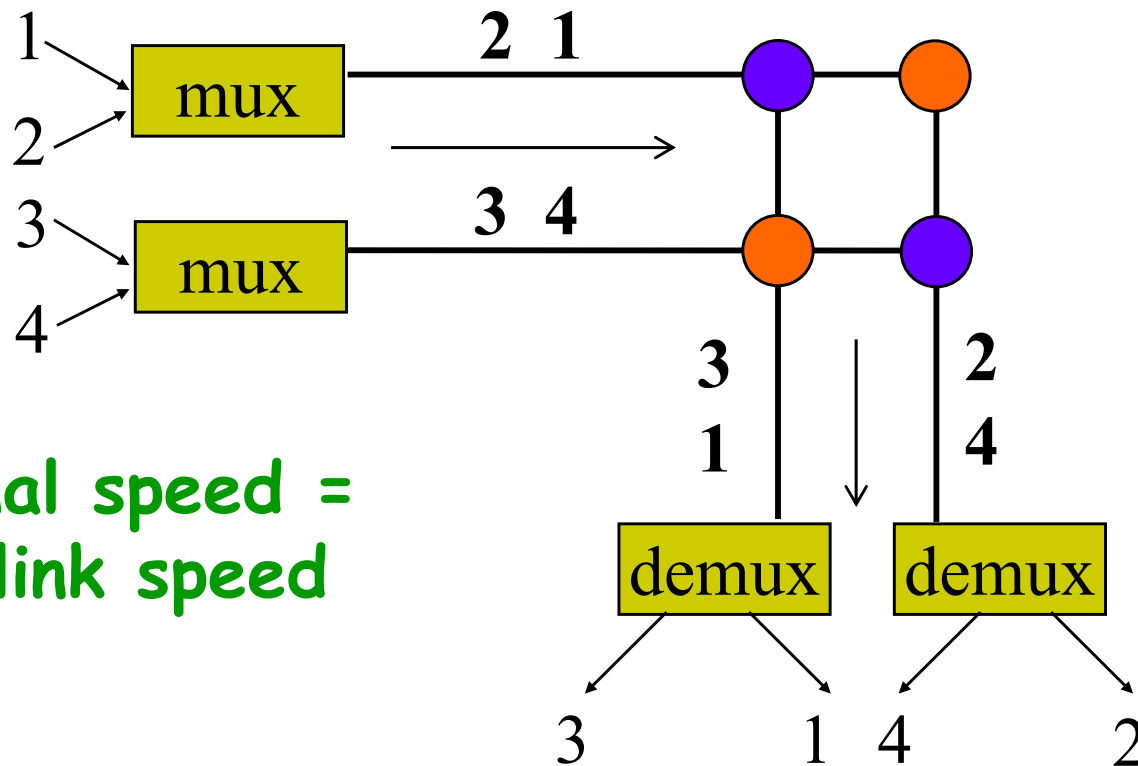
Crosspoints: 4
(down from 16)
memory speed: $\times 2$
(down from $\times 4$)

Time-Space: Example



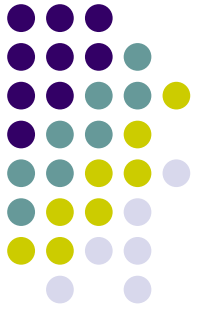
Target permutation: $1 \rightarrow 2, 2 \rightarrow 4,$
 $3 \rightarrow 1, 4 \rightarrow 3$

● time 1
● time 2

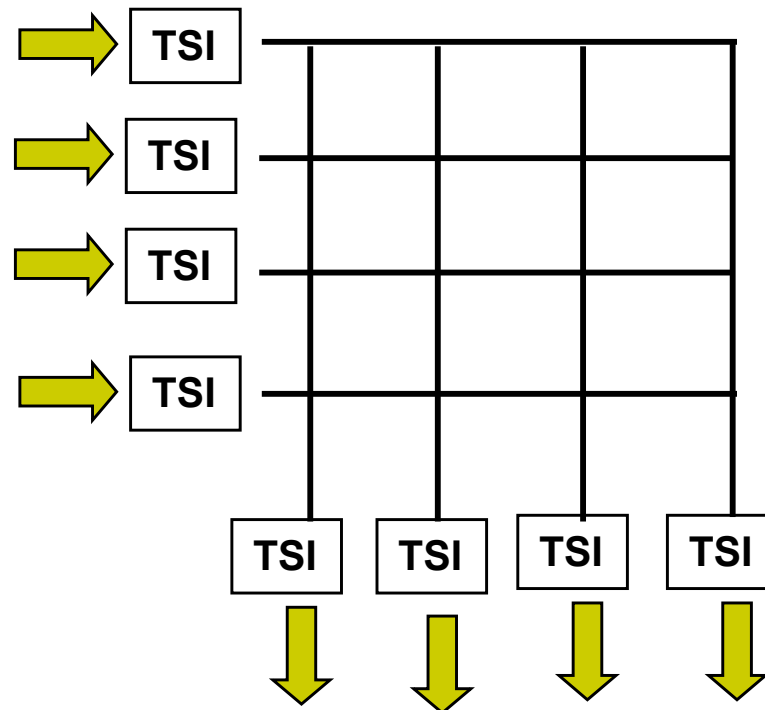


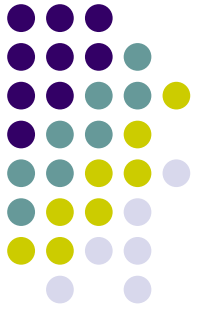
Internal speed =
twice link speed

Time-space-time (TST) switching



- Reorder samples both on input and output link
- Can retain original relative order



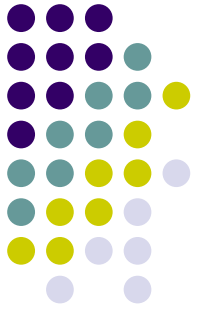


Need a schedule!

Definition of the problem.

- Input:
 - σ - a permutation of $\{1, \dots, n\}$
 - $\sigma(i) = j$ means input port i should go to output j
 - s - Size of crossbar side (crossbar is $s \times s$)
 - Implication: Each crossbar line handles $m = n/s$ calls
- Output: A schedule of length $t \geq m$ realizing σ
 - For each crossbar input line: calls order
 - For each time step : what permutation crossbar realizes

Finding a schedule

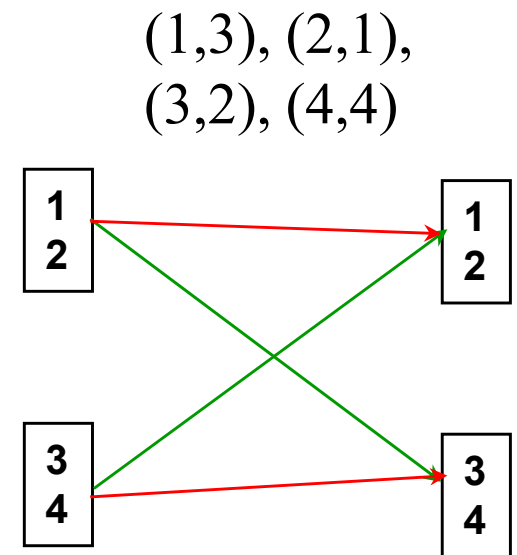


Goal: Minimize length of schedule

Model by a **routing graph**

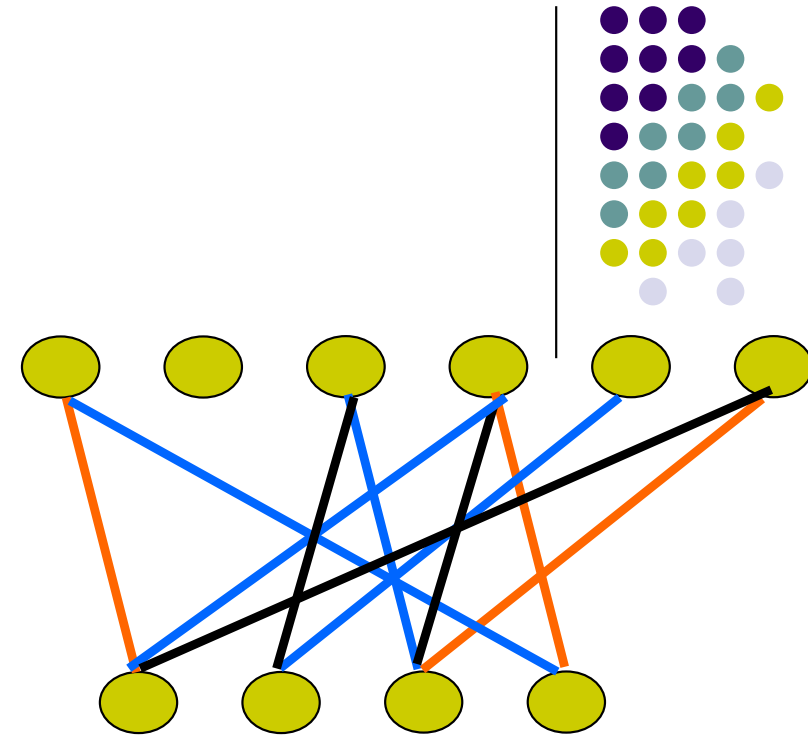
- a node for each crossbar input, a node for each crossbar output
- an edge connects nodes a, b if there's a call from a to b

Need to find good **edge coloring**



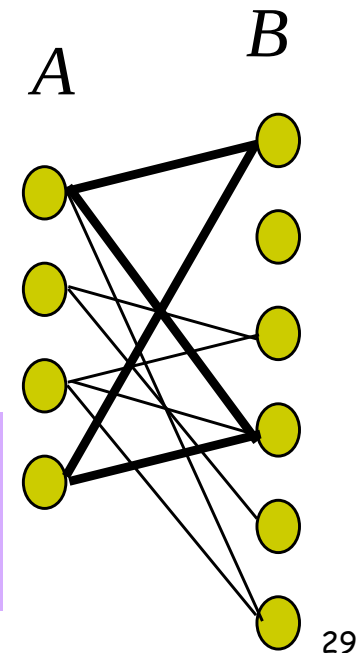
Some Graph Theory

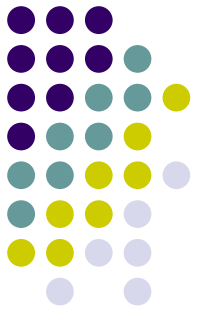
- **Edge coloring:**
Assignment of colors to edges,
so that no incident edges have
the same color.
Note: always $\# \text{colors} \geq \text{max degree}$.



- **Bipartite graph:** two sets of nodes, edges only
between sets (not inside a set).
 $G = (A, B, E)$, where $E \subseteq A \times B$.
- Matching for S : A set M of edges touching each
node in $S \subseteq A$, but not two edges in M intersect.

Hall's Theorem: In a bipartite graph (A, B, E) there's
a matching for $S \iff \forall X \subseteq S: |N(X)| \geq |X|$.





König's Theorem [1916]

The edges of any bipartite graph (A, B, E) with maximal degree Δ can be colored in Δ colors.

Proof: By induction.

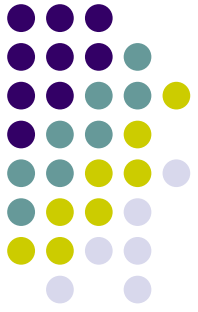
If $\Delta = 1$ the claim is trivial.

Otherwise, let A_Δ be the set of nodes in A with degree Δ . Clearly, every subset $X \subseteq A_\Delta$ has at least $|X|$ neighbors (otherwise there is a neighbor with degree $> \Delta$).

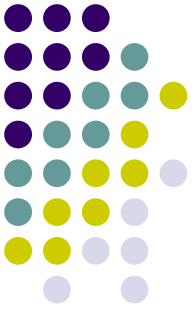
Hence there is a matching M for A_Δ . Color the edges of M by color " Δ ", remove edges

Apply induction, QED.

Corollary



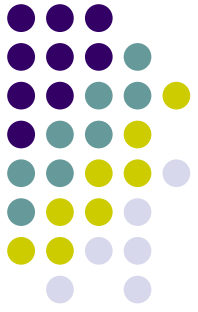
- Can find optimal schedule!
 1. Draw routing graph. Maximal degree $\Delta = m = \frac{n}{s}$
 2. Compute edge coloring using König's Theorem
 3. If **edge i - j colored c** , for $1 \leq c \leq \Delta$, then the schedule is "**connect port i to port j at step c** "
 - Optimality because length of schedule cannot be less than n/s , #calls per crossbar line
- Computation is a bit time consuming...
 - But needs to be done only at setup time.



General Space division

- Goal: extend the benefits of crossbars by better topology design
- Graph representation:
 - Nodes for inputs, outputs, and internal switches
 - Edges for links between them
- Routing:
 - edge disjoint paths
- Cost:
 - number of crosspoints
 - VLSI Layout area

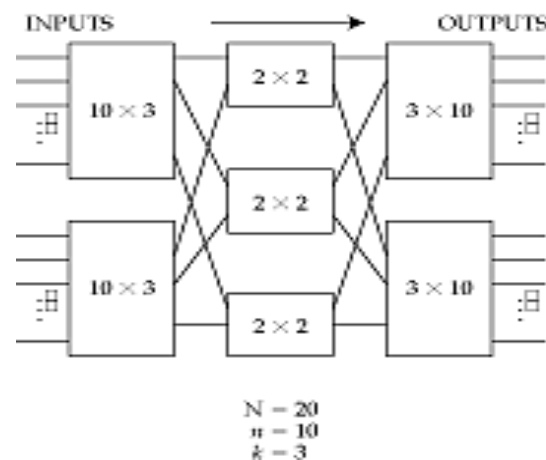
Multistage switch



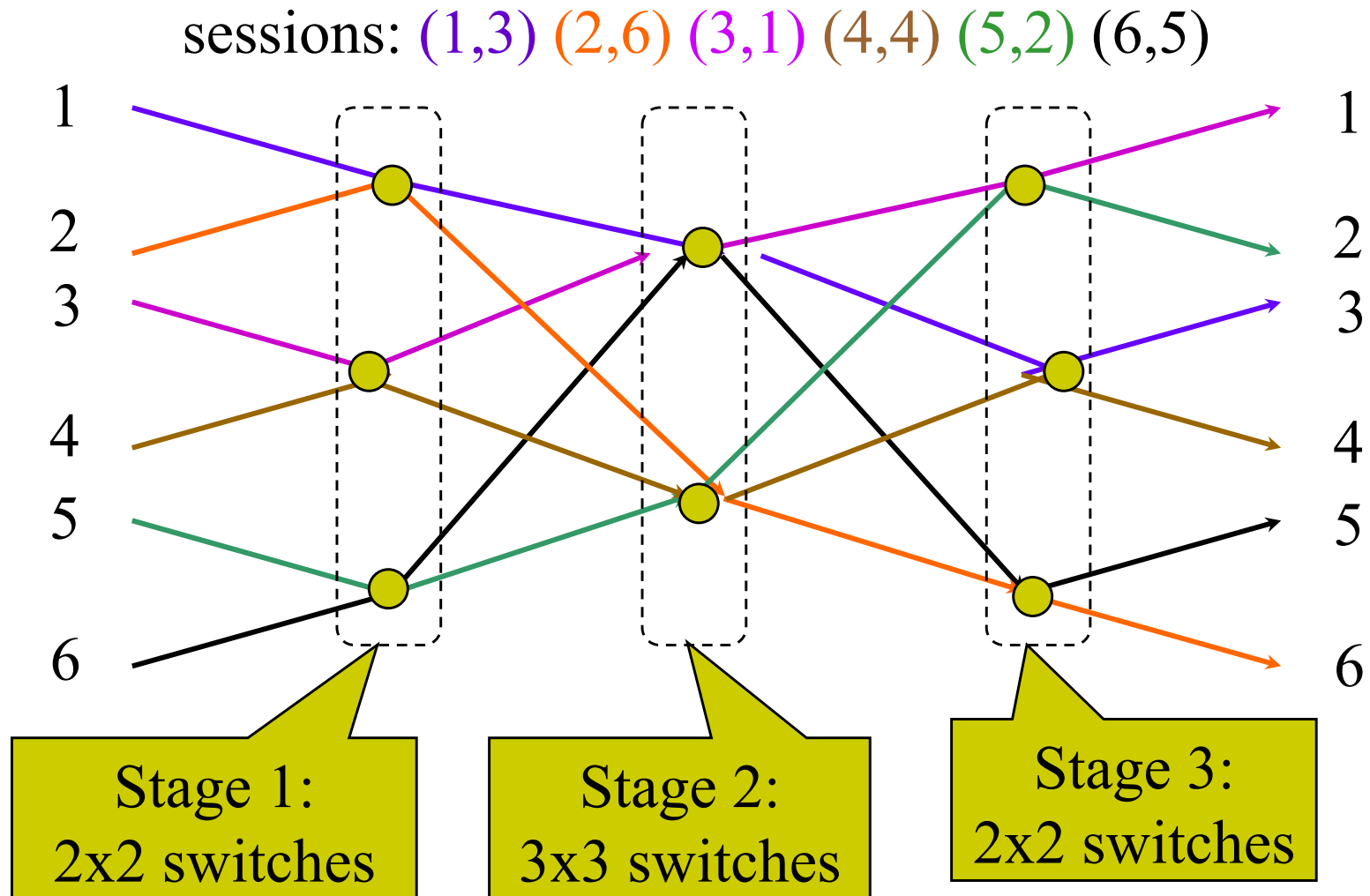
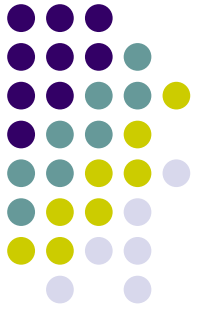
- In a crossbar, calls go through a single crosspoint (internal switch)

Multistage switch basic idea:

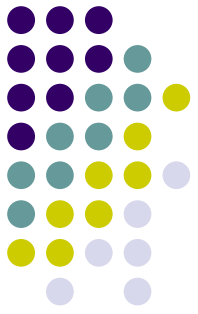
Allow calls to go through more than one internal switch



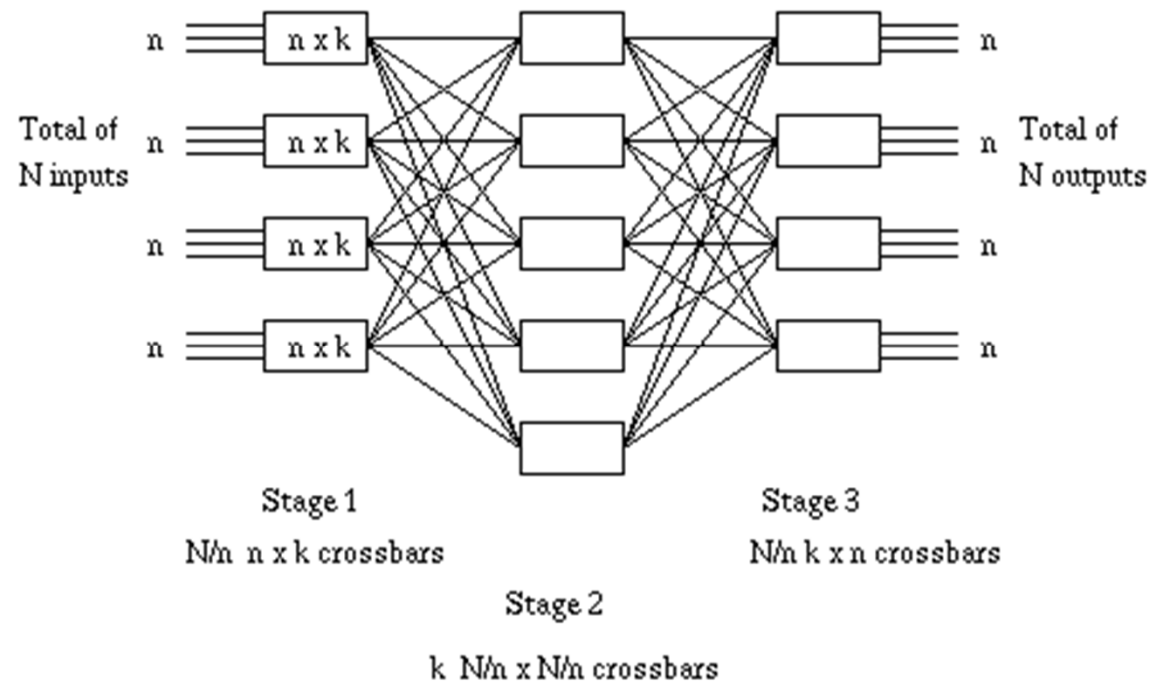
Example



Clos networks: (N, n, k)

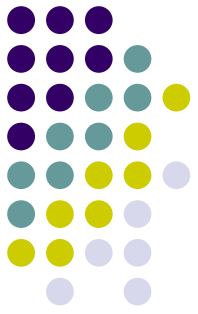


- Describing $N \times N$ switch
- Three layers (stages):
All in layer 1 and 3 connected to all in layer 2
- Layers 1,3: $n \times k$ switches (N/n of them)
- Layer 2: k switches, each is $N/n \times N/n$



- # crosspoints: $2 \frac{N}{n} \cdot nk + k \left(\frac{N}{n} \right)^2 = 2Nk + \frac{kN^2}{n^2}$
- This is less than N^2 for $k < \frac{N}{2 + N/n^2}$

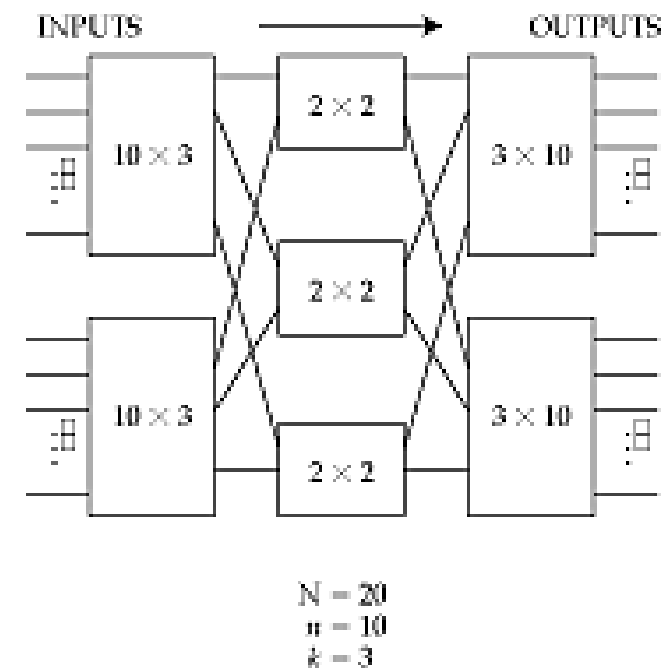
Clos Networks



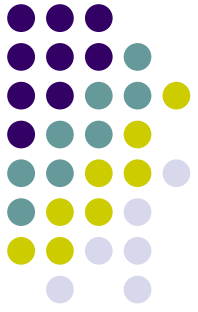
Can suffer internal blocking.
E.g., if $k < n$ because bandwidth between stages is $k \cdot N/n$.

Types of non-blocking:

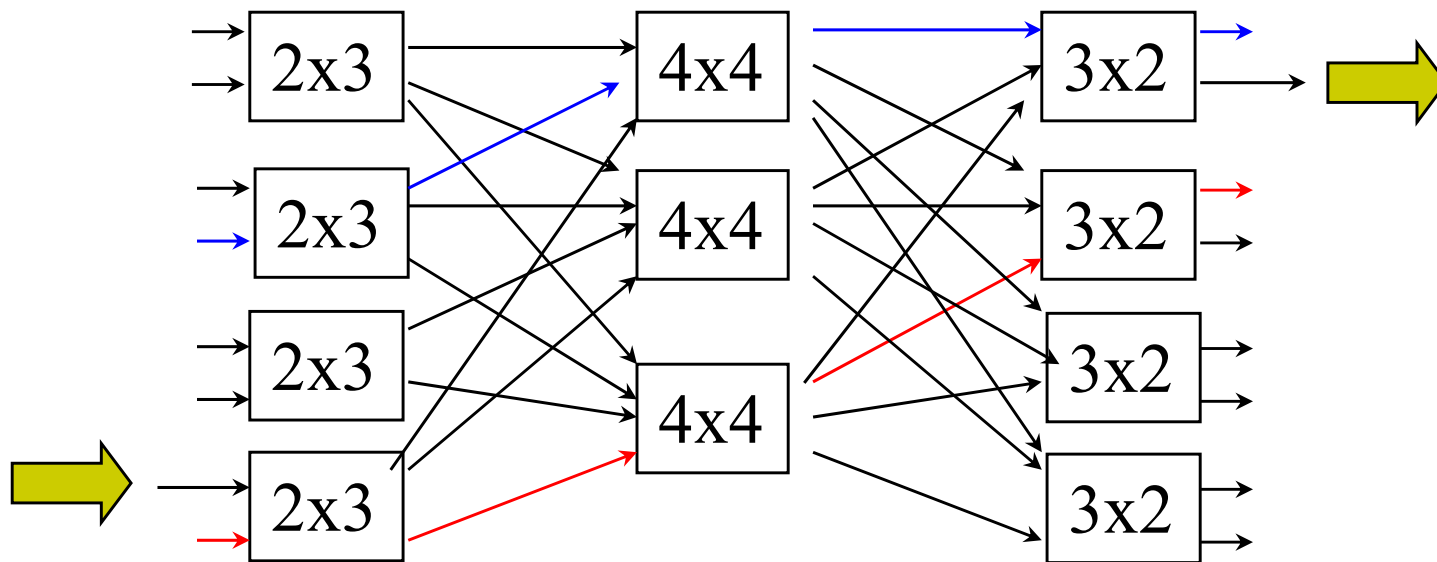
- **Strict sense:** can find a route from a free input to a free output without changing existing routes
- **Rearrangable:** can route any set of pairs (destroying existing state)



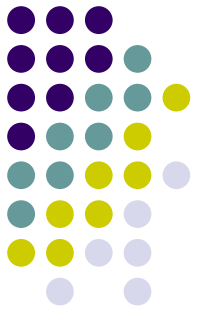
Example



- New call from input 7 to output 2



$N=8$
 $n=2$
 $k=3$



Clos Theorem [1953]

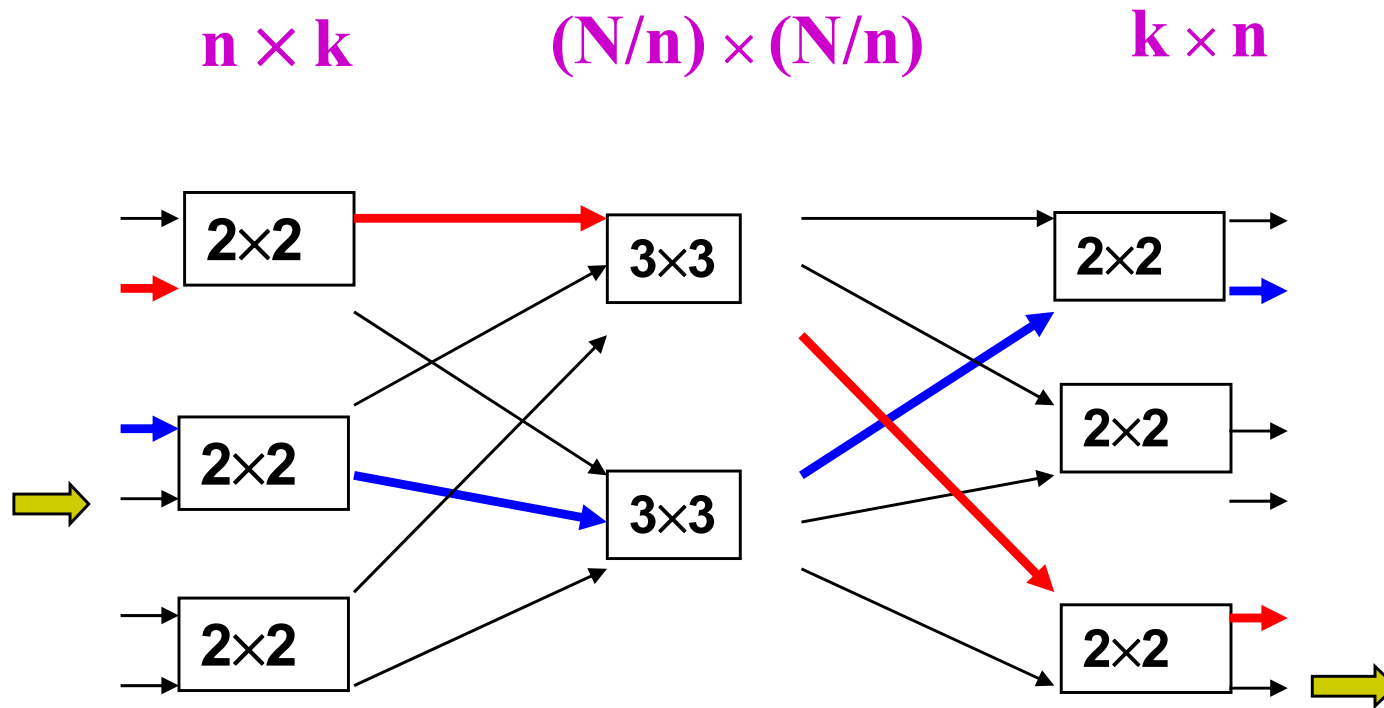
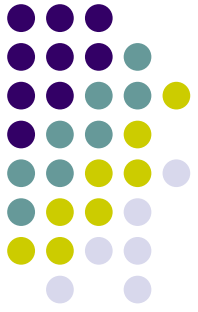
Clos network is strict-sense nonblocking iff $k \geq 2n - 1$.

Proof: If $k \geq 2n - 1$: Consider a call from sub-switch a in stage 1 to sub-switch b in stage 3.

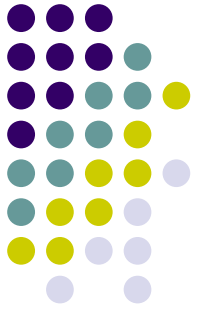
connected calls at a and b is at most $(n - 1)$ each, hence exists a free stage-2 switch to connect a, b .

If $k < 2n - 1$: From a sub-switch a in stage 1, connect n calls. Must use n different level 2 sub-switches. Then can't connect $n - 1$ calls from a stage 3 sub-switch b to layer 1 sub-switches (a excluded).

Blocking Example (strict sense)



$N=6$
 $n=2$
 $k=2$



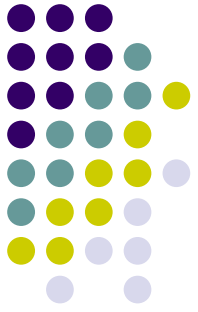
Rearrangeable Clos networks

Theorem [Slepian,Duguid]: Clos (N, n, k) is rearrangeably non-blocking (RNB) iff $k \geq n$.

Proof: To be RNB must have $k \geq n$: consider N calls. Bandwidth between layers is kN/n .

To show $k \geq n$ implies RNB: build a bipartite routing graph:

- Nodes: layer 1 switches, layer 3 switches
- Edges: calls (input node --- output node)
- max degree $\leq n$, so by König's Theorem, can color edges in n colors.
- Assign 2nd stage switch to each color (coloring = routing!)
- Since $k \geq n$, we're done.



Recursive construction: Benes

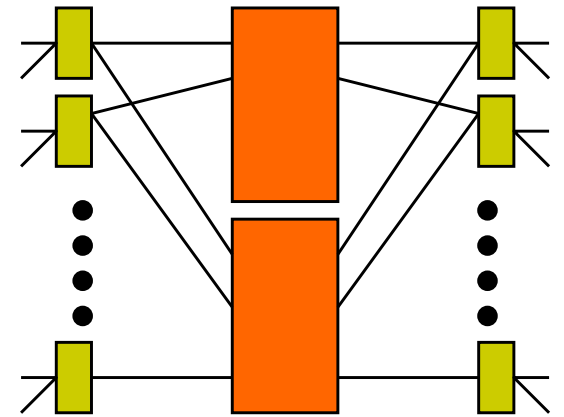
- First and last stage $2 \times N/2$ crossbars 
- Middle stage: $N/2 \times N/2$ switch 
 - recursive construction!
- #crosspoints:

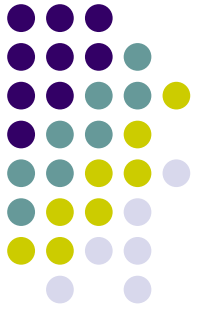
$$X(N) = 2 \frac{N}{2} \cdot 4 + 2X\left(\frac{N}{2}\right)$$

$$= 4N + 2X\left(\frac{N}{2}\right)$$

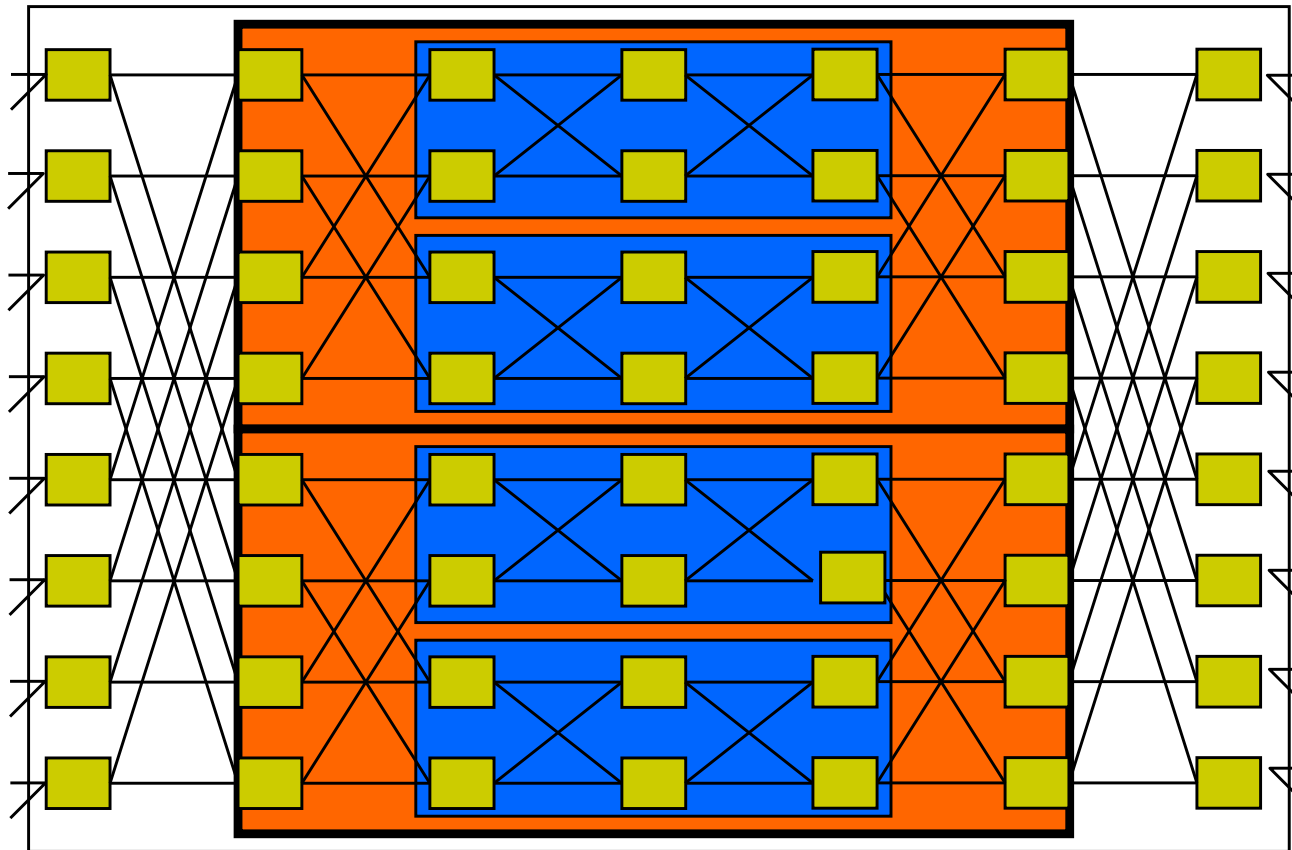
$$= 2N + 2 \left(4 \frac{N}{2} + 2X\left(\frac{N}{4}\right) \right)$$

$$= \dots = 4N \log N - 2N$$





16×16 Benes network

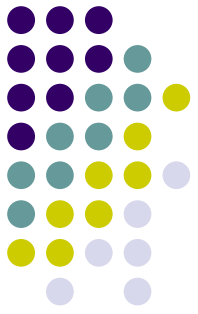


Benes network (back-to-back *Butterfly*)

#stages: $2\log_2 N - 1$

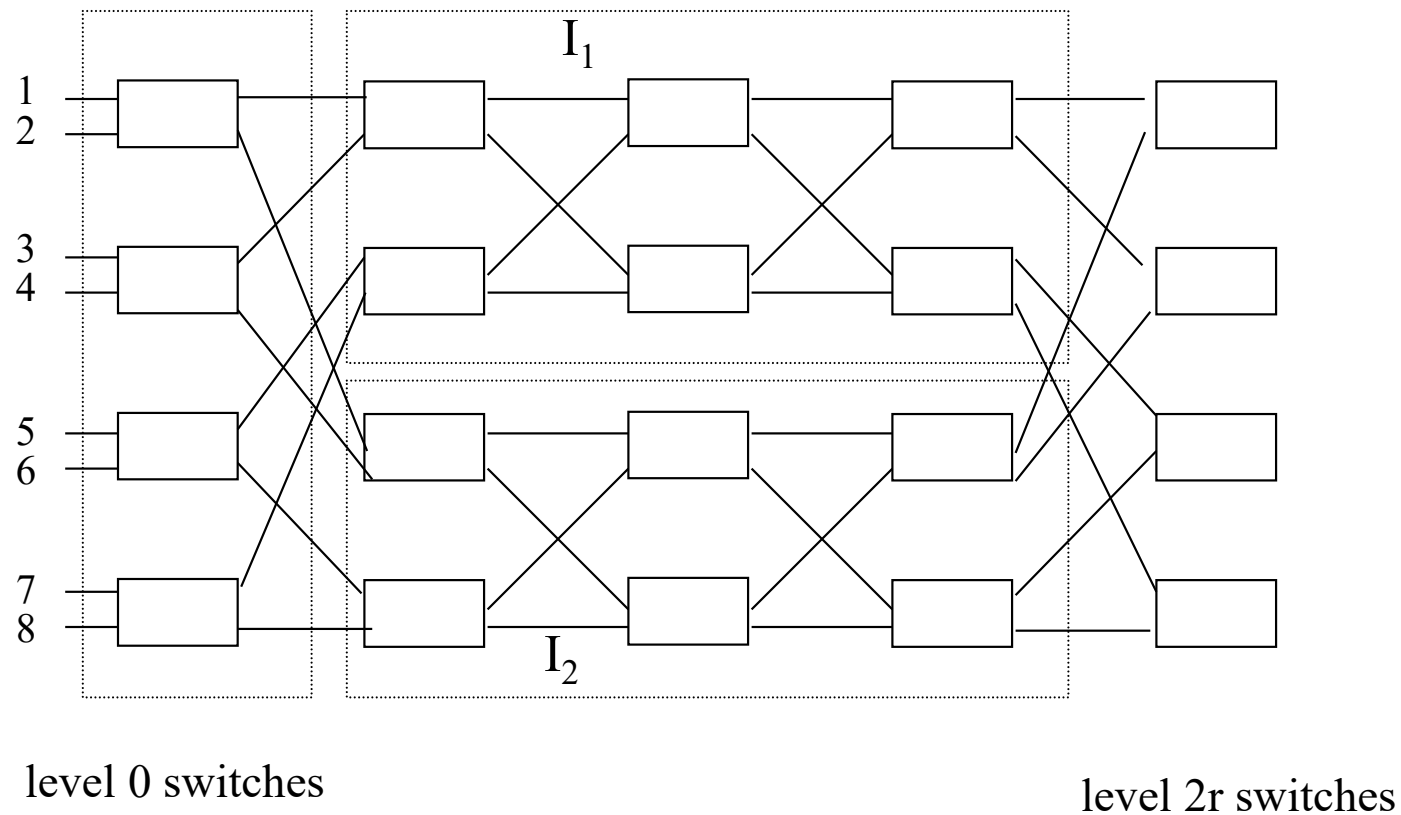
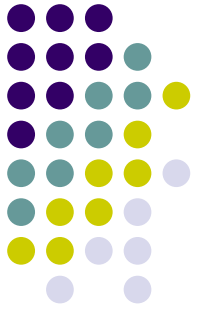
#cross points : $4N\log_2 N - 2N$

Greedy permutation routing



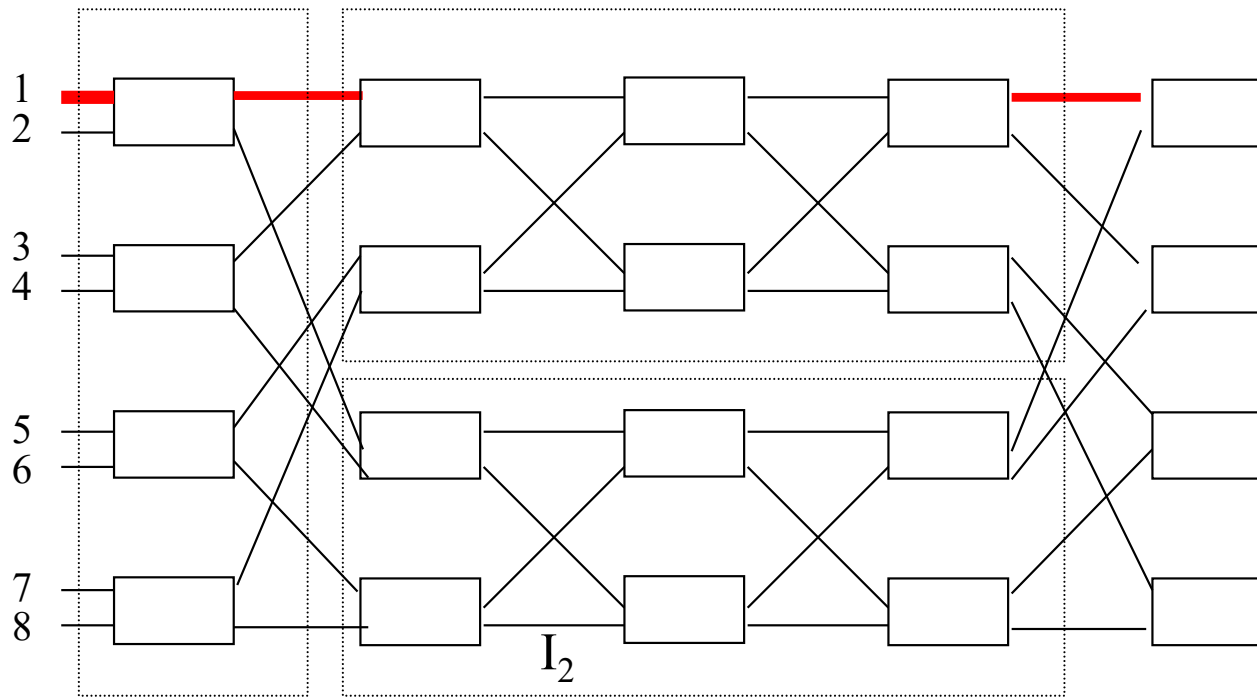
- Sufficient to choose whether to go “up” or “down” at each intermediate node (solve for the upper and lower intermediate subswitches recursively).
- Idea: go back and forth, alternating up and down.
 - Start from input i_1 , choose “up”. Get to its output o_1 . In the same output box, there're output o_2 .
 - From o_2 , go back, choosing “down” (up is taken!)
 - Continue until completing a cycle, and then start with remainder.

Example: Benes Network for $r=2$



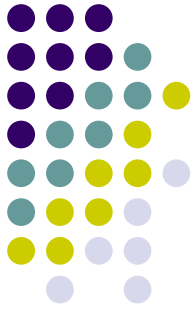
Example

$$\left(\begin{array}{cccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 1 & 5 & 6 & 8 & 4 & 2 & 3 & 7 \end{array} \right)$$



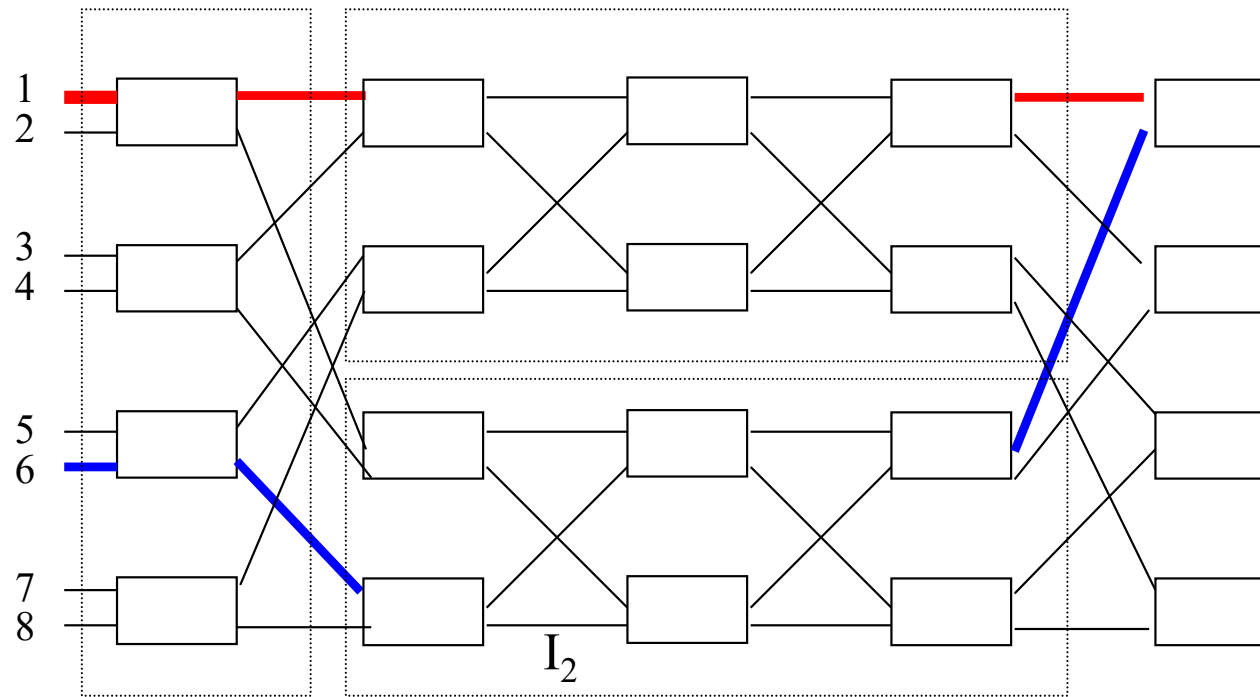
level 0 switches

level 2r switches



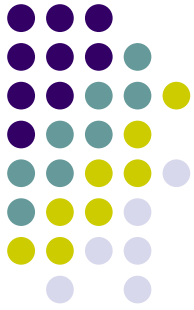
Example

$$\left(\begin{array}{cccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 1 & 5 & 6 & 8 & 4 & 2 & 3 & 7 \end{array} \right)$$



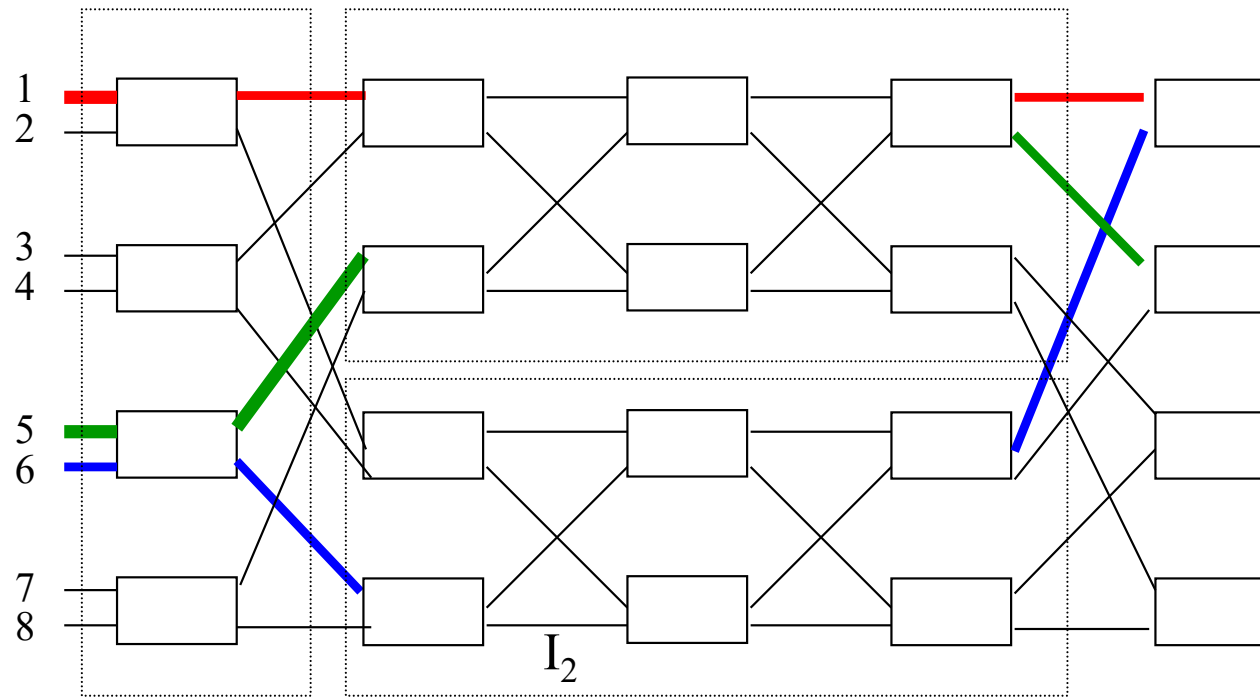
level 0 switches

level 2r switches



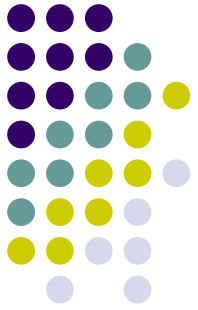
Example

$$\left(\begin{array}{cccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 1 & 5 & 6 & 8 & 4 & 2 & 3 & 7 \end{array} \right)$$



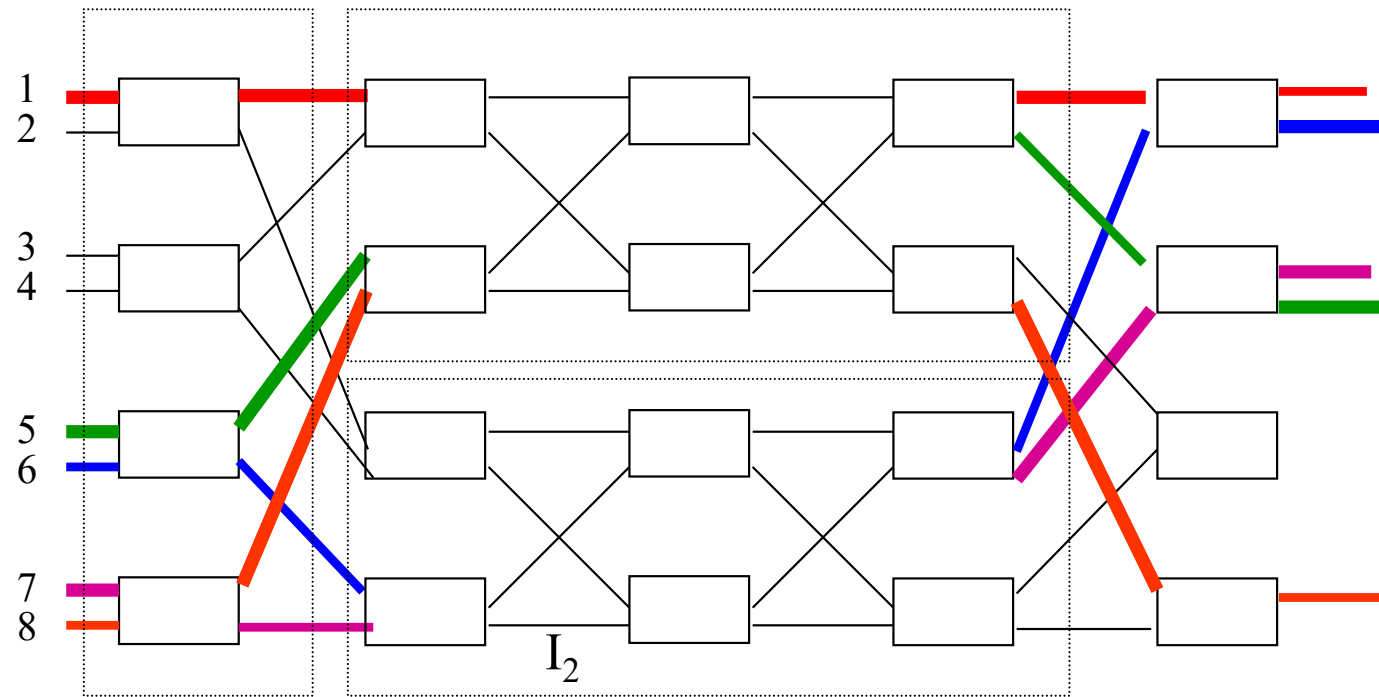
level 0 switches

level 2r switches



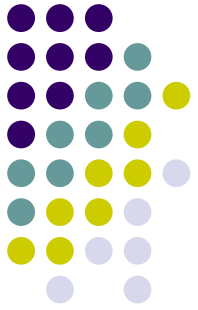
Example

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 1 & 5 & 6 & 8 & 4 & 2 & 3 & 7 \end{pmatrix}$$

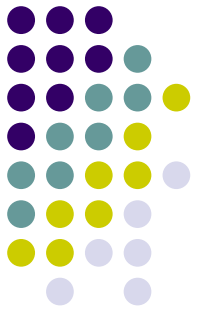


level 0 switches

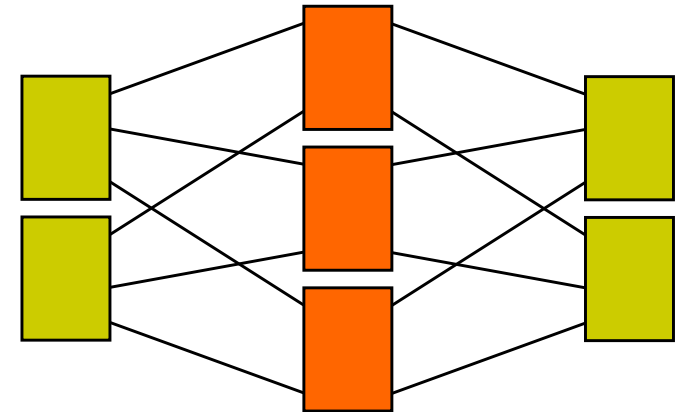
level 2r switches

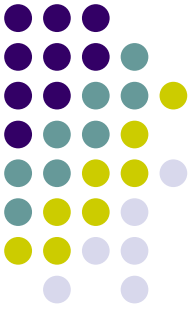


Recursive strictly non-blocking Clos Networks



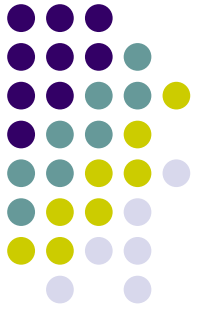
- #crosspoints: if $k = 2n - 1$, choose $n = \sqrt{N/2}$, get cost of $O(N^{1.5})$ points.
- Recursive construction: first and last stage $\sqrt{N}$ subswitches, each $\sqrt{N} \times 2\sqrt{N}$; middle stage $2\sqrt{N}$ subswitches of size $\sqrt{N} \times \sqrt{N}$.
- $$\begin{aligned} X(N) &\approx 2\sqrt{N} \cdot 2X(\sqrt{N}) + 2\sqrt{N} \cdot X(\sqrt{N}) \\ &= 6\sqrt{N} \cdot X(\sqrt{N}) \\ &= 6N^{1/2} 6N^{1/4} 6N^{1/8} \dots 6N^{1/\log N} \\ &\leq 6^{\log \log N} N \\ &= (\log N)^{\log 6} N \approx N(\log N)^{2.58} \end{aligned}$$





Fabrics: Summary

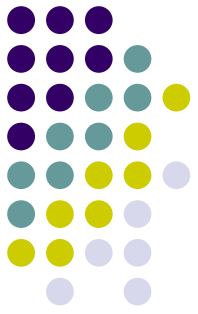
- Rearrangeably non-blocking networks can be constructed using $O(N \log N)$ crosspoints.
- Strictly non-blocking networks can be practically constructed in $O(N \log^2 N)$ crosspoints (Batcher-Banyan, Cantor)
 - Theoretical construction: $O(N \log N)$ crosspoints for strictly non-blocking. Non-practical due to large hidden constants (based on AKS sorting network)



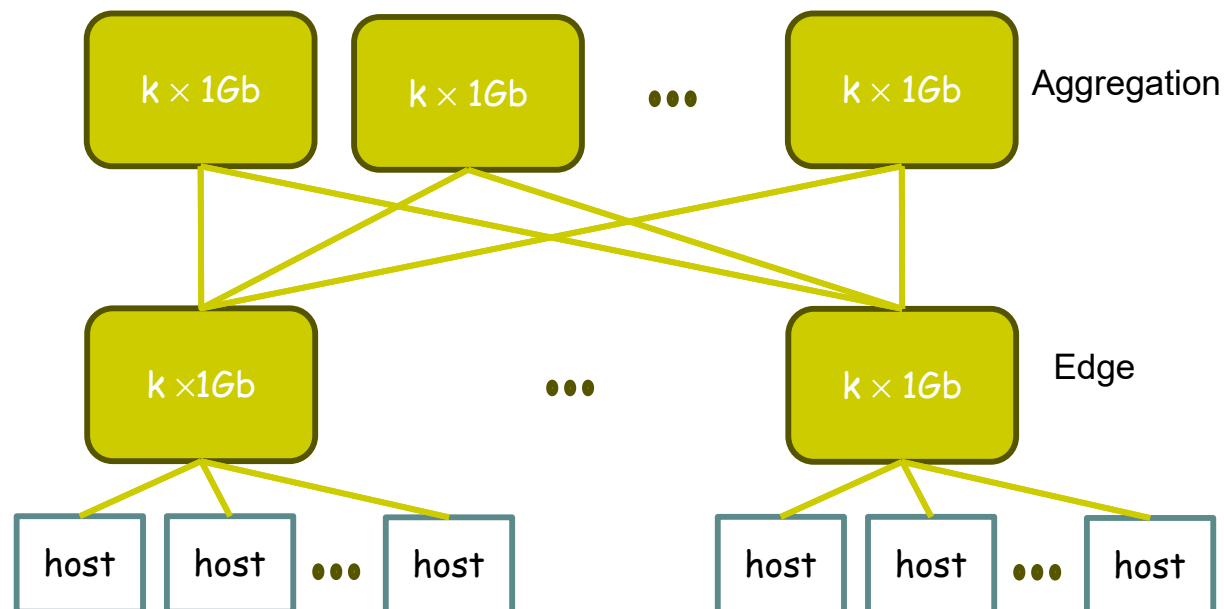
New Concepts

- Switch fabric
- Timeslot interchange (TSI)
- Time-Space switching (TS)
- Koenig's Lemma and TS scheduling
- Crossbar
- Clos networks (N, n, k)
- Non-blocking switch: strict-sense and rearrangeable
- Construction: Benes network
- Construction: Batcher sort

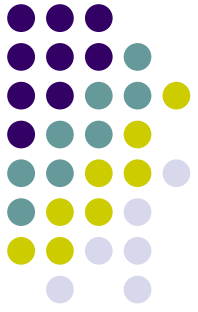
Datacenter architecture: Clos (a.k.a. Fat Tree)



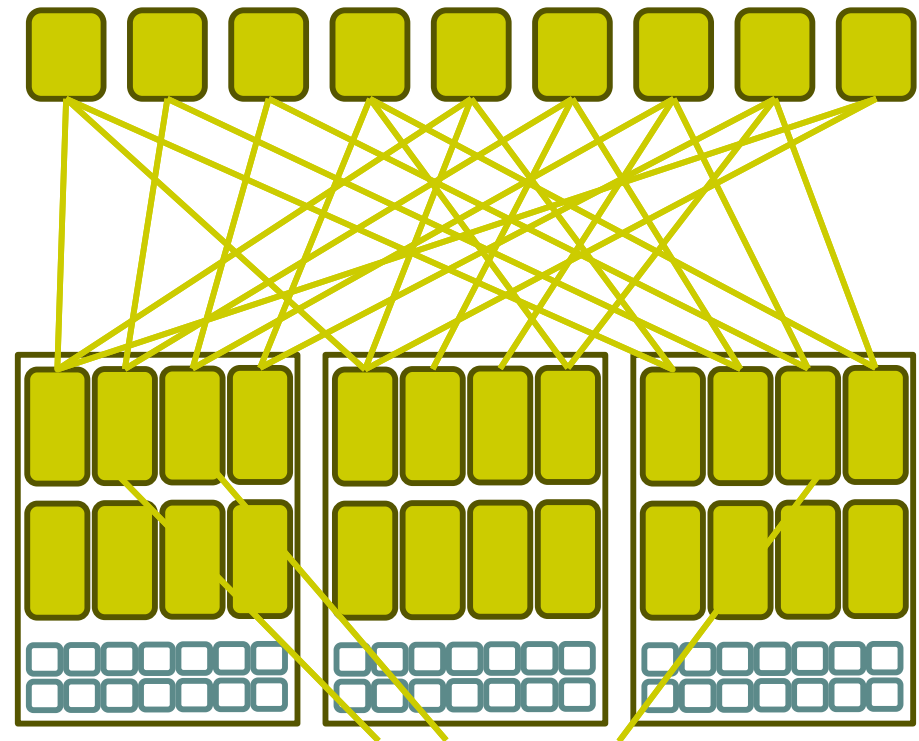
- Use only k -port 1Gbps switches
- Subunit: pod.
 - $k/2$ hosts connected to each edge switch.
 - $k/2$ edge switches in a pod
 - $k/2$ aggregation switches in a pod
- Each edge switch is connected to all aggregation switches
- $k^2/4$ hosts, k switches



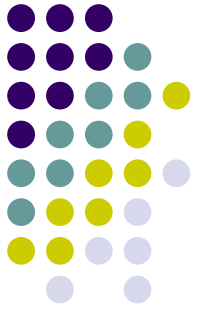
Datacenter architecture: connecting the pods



- k pods, $\frac{k^2}{4}$ core switches
- Each aggregation switch is connected to $k/2$ core switches
- Each pod to all $k^2/4$ core switches
- Each core switch is connected to all pods
 - Port i to pod i



Datacenter architecture: numbers



- # hosts: $k \cdot \frac{k^2}{4} = \frac{k^3}{4}$
 - 27,648 hosts for $k = 48$
- $k^2 + \frac{k^2}{4}$ switches (2880 for $k = 48$)
- Power: less than 250kW
- Total cost (2008): less than \$8.7m